PRACTICAL 4

Tashil Haripersad (u25466004)
Graeme Geerkens (u24566285)
Nathan Spavins (u25054245)

Task 1: Event concept and complete architecture

Domain and problem

A freight forwarder manages **shipments**: nested groupings of cargo that must be inspected in different ways, whose individual boxes move through a customs/delivery lifecycle, and which sometimes need extra handling (insurance, refrigeration) bolted on after the fact – sometimes several kinds of handling stacked on the same box. A shipment is a tree: a global shipment contains **containers**, containers contain **pallets**, pallets contain **boxes** (and, in principle, further nested groupings) – so treating "one box" and "a whole pallet" uniformly when asking "how much does this weigh?" or "how much will this cost to ship?" is genuinely useful, not just a textbook excuse for polymorphism.

GoF participants

Composite

Component	Shippable
Leaf (concrete)	Box
Composite	ShippingContainer
Client	main.cpp

Iterator

Aggregate	CargoAggregate
ConcreteAggregate	ShippingContainer
Iterator	CargoIterator
ConcreteIterator	FullInventoryIterator, HazardousMaterialsIterator

State

Context	Box
State	BoxState
ConcreteState	InWarehouseState, InTransitState, CustomsClearanceState, DeliveredState

Decorator

Component	Shippable
ConcreteComponent	Box
Decorator	ShippableDecorator
ConcreteDecorator	InsuredShipping, RefrigeratedShipping

Design Rationale

ShippingContainer::getWeight() / getCost() sum over their children, recursing into any nested ShippingContainer automatically because every child is accessed only through the Shippable interface.

Both concrete iterators are built from the same Shippable::collectLeaves traversal hook, but apply a different **selection rule**: FullInventoryIterator keeps everything, HazardousMaterialsIterator keeps only items where isHazardous() is true. Two independent iterators can be live over the same ShippingContainer at once (demonstrated in main.cpp, Scenario 2) because each one owns its own snapshot list and position counter — they share no mutable state.

Box never contains an if (state == ...) chain: it just forwards loadOntoTruck() / arriveAtCustoms() / clearCustoms() to whichever BoxState it currently holds, and that state decides whether the transition is legal. An illegal transition throws InvalidStateTransition (a std::logic_error), which main.cpp catches and reports — the "sensible handling" the brief asks for, rather than a silent no-op or a crash.

ShippableDecorator forwards every Shippable operation to the object it wraps by default; each concrete decorator overrides only what it changes (InsuredShipping adds a premium to getCost(); RefrigeratedShipping adds insulation weight to getWeight() and a surcharge to getCost()). Because the Composite pattern's Component and the Decorator pattern's Component are the *same* interface (Shippable), a decorated Box — even one wrapped twice, as BX-1003 is in the demo — can sit inside a ShippingContainer and be picked up by either iterator with no special casing anywhere in the system.

Ownership and destruction policy

Every owning relationship uses a raw pointer plus an explicit destructor that deletes what it owns, and disables copying so ownership can never become ambiguous or shared:

- A ShippingContainer owns its children exclusively via std::vector<Shippable*>. ~ShippingContainer() loops over that vector and deletes every child, so destroying a container recursively destroys everything beneath it, including further nested containers (whose own destructors then do the same thing one level down). Its copy constructor and copy assignment operator are = deleted — copying a container by value would otherwise produce two containers whose children_ vectors point at the same objects, and both destructors would try to delete them.
- A Box owns its current BoxState*; transitioning state (Box::setState) deletes the old state object before installing the new one, and ~Box() deletes whatever state is left. Box's copy operations are likewise deleted.
- A ShippableDecorator owns the Shippable* it wraps, deleting it in ~ShippableDecorator(), so a stack of decorators forms a single ownership chain (outermost decorator → ... → innermost Box) with no aliasing. Its copy operations are deleted too; every concrete decorator inherits that restriction automatically.
- There is exactly one owner for every object at every point in time. To move an item between containers, code must explicitly call releaseItem(name) on the source (which removes the pointer from children_ *without* deleting it and hands it back to the caller) and then addItem(...) on the destination — there is never a moment where two containers hold the same object.
- releaseItem only searches **direct** children, deliberately — reaching deep inside a nested group "merely to find something" is exactly what the brief prohibits; a caller that needs to act on a

deeply-nested box gets there by holding (or being handed) a pointer to the specific container that owns it, which is how main.cpp does it.

- The two CargoIterator factory methods (createFullIterator(), createHazardousIterator()) are the one place the project deliberately hands back a raw, caller-owned pointer rather than storing it inside another owner: iterators are meant to be short-lived, throwaway objects, so main.cpp deletes each one as soon as it is finished with it (see the delete full;, delete hazmat;, delete inFlight; etc. calls).

Task 3: Dynamic Behaviour and Design Decisions

Traversal-modification policy

Policy: snapshot iteration. Both FullInventoryIterator and HazardousMaterialsIterator compute their full sequence of Shippable* pointers once, in their constructor, by calling collectLeaves on the container they were built from. After that point they are entirely independent of the container's internal storage — they just walk their own std::vector<Shippable*>.

Consequences, demonstrated in main.cpp Scenario 2:

- An iterator created *before* a structural change (moving BX-2001 between containers), a decoration change (re-wrapping BX-1002 in InsuredShipping), and an addition (BX-2002) continues to report exactly the objects that existed when it was created, in the same order, even though the underlying hierarchy has since changed shape.
- A *new* iterator created afterwards reflects the updated hierarchy (five items instead of four; BX-1002 now shows its [Insured] tag and updated cost; BX-2001 now appears under Pallet A1 instead of Container MSCU-2233).
- This is safe by construction — an in-progress traversal can never be invalidated by another part of the system reshaping the tree, and never observes a torn/half-updated view — at the cost of that traversal not reflecting the very latest state. That trade-off is the deliberate, documented choice for this system, and is why iterators are cheap, short-lived, throwaway objects here rather than long-lived cursors into live container state.

Traversal without exposing internals

ShippingContainer never exposes its vector<Shippable*>. Client code only ever gets a CargoIterator from createFullIterator() / createHazardousIterator(), and only ever calls hasNext() / next() on it. The recursive walk itself happens inside Shippable::collectLeaves, which is a traversal-support hook rather than a general-purpose accessor — a ShippingContainer recurses into its children, a Box or ShippableDecorator appends itself, and neither exposes storage.

Task 5: Debugging and Memory Investigation

Built with:

g++ -std=c++11 -Wall -Wextra -g -c <each .cpp> -o <each .o>

g++ -std=c++11 -Wall -Wextra -g -o taskforge <all .o>

-Wall -Wextra produced zero warnings.

GDB evidence: breakpoints, stepping, and state inspection

Breakpoint on FullInventoryIterator's constructor, stepping through the recursive collectLeaves call that builds its snapshot vector.

```
(gdb) break FullInventoryIterator::FullInventoryIterator
(gdb) run
Breakpoint 1, FullInventoryIterator::FullInventoryIterator (this=0x555555577820, root=...) at
FullInventoryIterator.cpp:4
4       FullInventoryIterator::FullInventoryIterator(ShippingContainer& root) : position_(0) {
(gdb) print root.getName()
$1 = "Global Shipment GS-01"
(gdb) next
5           root.collectLeaves(items_);
(gdb) print items_.size()
$2 = 0
(gdb) next
6       }
(gdb) print items_.size()
$3 = 4
(gdb) print items_[0]->getName()
$4 = "BX-1001 Electronics Crate"
(gdb) print items_[1]->getName()
$5 = "BX-1002 Industrial Chemicals"
(gdb) print items_[2]->getName()
$6 = "BX-1003 Frozen Seafood [Refrigerated @ -18C] [Insured]"
(gdb) print items_[3]->getName()
$7 = "BX-2001 Lithium Batteries"
```

items_ is empty (size() == 0) immediately before the call to collectLeaves, and contains exactly four pointers immediately after. This confirms the snapshot is built once, at construction time, by recursing through the Composite structure (ShippingContainer::collectLeaves recursing into children, each Box/ShippableDecorator appending itself).

A genuine bug: symptom, cause, debugging evidence, correction

Symptom

The line that frees the previous BoxState before installing the new one was missing from box::setState:

```
// buggy version
void Box::setState(BoxState* newState) {
   // delete state_;   <-- missing
   state_ = newState;
}
```

The program's output was completely unaffected: ./taskforge printed byte-for-byte the same manifests, lifecycle transitions, and totals as the correct version. The only way to catch it was memory instrumentation:

```
==973== HEAP SUMMARY:
==973==     in use at exit: 24 bytes in 3 blocks
==973==   total heap usage: 107 allocs, 104 frees, 82,373 bytes allocated
==973==
==973== 8 bytes in 1 blocks are definitely lost in loss record 1 of 3
==973==    at 0x4846FA3: operator new(unsigned long)
==973==    by 0x10B5D7: Box::Box(...) (Box.cpp:8)
==973==    by 0x10FF6A: main (main.cpp:91)
==973==
==973== 8 bytes in 1 blocks are definitely lost in loss record 2 of 3
==973==    at 0x4846FA3: operator new(unsigned long)
==973==    by 0x10C994: InWarehouseState::loadOntoTruck(Box&) (BoxState.cpp:7)
==973==    by 0x10B7AF: Box::loadOntoTruck() (Box.cpp:19)
==973==    ...
==973==
==973== 8 bytes in 1 blocks are definitely lost in loss record 3 of 3
==973==    at 0x4846FA3: operator new(unsigned long)
==973==    by 0x10CD0A: InTransitState::arriveAtCustoms(Box&) (BoxState.cpp:21)
==973==    by 0x10B7E9: Box::arriveAtCustoms() (Box.cpp:20)
==973==    ...
==973==
==973== LEAK SUMMARY:
==973==    definitely lost: 24 bytes in 3 blocks
```

Three BoxState objects (8 bytes each, a single vtable pointer, no other members), exactly matching the three transitions BX-1001 makes in Scenario 1 (InWarehouse -> InTransit -> CustomsClearance -> Delivered). Each stack trace points at a new inside a BoxState::loadOntoTruck/arriveAtCustoms override or the Box constructor: every state object that was ever replaced leaked, while the final DeliveredState (never replaced, cleaned up by ~Box()) did not.

Cause: found with GDB

Valgrind shows where memory was allocated, not why it was never freed, so GDB was used on Box::setState to watch the pointer swap happen live:

```
(gdb) break Box::setState
(gdb) run
Breakpoint 1, Box::setState (this=0x5555555774b0, newState=0x555555577850) at Box.cpp:26
26        state_ = newState;
(gdb) print state_
$1 = (BoxState *) 0x555555577530
(gdb) print state_->name()
$2 = "InWarehouse"
(gdb) print newState->name()
$4 = "InTransit"
(gdb) next
27    }
(gdb) print state_
$5 = (BoxState *) 0x555555577850
```

```
(gdb) continue
Breakpoint 1, Box::setState (this=0x5555555774b0, newState=0x555555577cc0) at Box.cpp:26
(gdb) print state_
$7 = (BoxState *) 0x555555577850
(gdb) print state_->name()
$8 = "InTransit"
(gdb) print newState->name()
$9 = "CustomsClearance"
```

Root cause: at the moment setState is entered, state_ (for example 0x555555577530, InWarehouseState) is a live, heap-allocated object with no other pointer referencing it anywhere in the program. The very next line overwrites that pointer with newState. No delete occurs between reading the old value and discarding it, so 0x555555577530 becomes unreachable the instant state_ = newState; executes, confirmed twice in a row across two consecutive transitions.

Correction

```
// fixed version
void Box::setState(BoxState* newState) {
  delete state_;
  state_ = newState;
}
```

```
==1024== HEAP SUMMARY:
==1024==     in use at exit: 0 bytes in 0 blocks
==1024==   total heap usage: 107 allocs, 107 frees, 82,373 bytes allocated
==1024==
==1024== All heap blocks were freed -- no leaks are possible
==1024== ERROR SUMMARY: 0 errors from 0 contexts (suppressed: 0 from 0)
```

stdout was diffed against the pre-fix run and is byte-for-byte identical, confirming the fix changes only ownership behaviour, not program logic.

Valgrind evidence for the final application

```
valgrind --leak-check=full --show-leak-kinds=all --track-origins=yes ./taskforge
```

```
==903== HEAP SUMMARY:
==903==     in use at exit: 0 bytes in 0 blocks
==903==   total heap usage: 107 allocs, 107 frees, 82,373 bytes allocated
==903==
==903== All heap blocks were freed -- no leaks are possible
==903==
==903== For lists of detected and suppressed errors, rerun with: -s
==903== ERROR SUMMARY: 0 errors from 0 contexts (suppressed: 0 from 0)
```

Zero errors, zero leaks of any kind, and every one of the 107 allocations made across both demo scenarios is paired with exactly one delete, consistent with the single-owner policy described in README section 3.

Task 6: Docker and GitHub Workflow

https://github.com/tashil-haripersad/TaskForge.git

Workflow Reflection and Team Contribution Statement

All three team members contributed substantially and roughly equally to this submission. Tashil designed and implemented the Composite pattern foundation, integrated all branches into a working main.cpp, README.md, and Makefile, and produced the UML class diagram and final PDF. Graeme designed and implemented the Iterator pattern, produced the object and state diagrams, and carried out the GDB/Valgrind debugging investigation. Nathan designed and implemented the State and Decorator patterns, produced the activity diagrams, and configured the Docker build/test environment. All members reviewed each other's pull requests and participated in the design decisions (ownership model, traversal policy) that affected the system as a whole.